

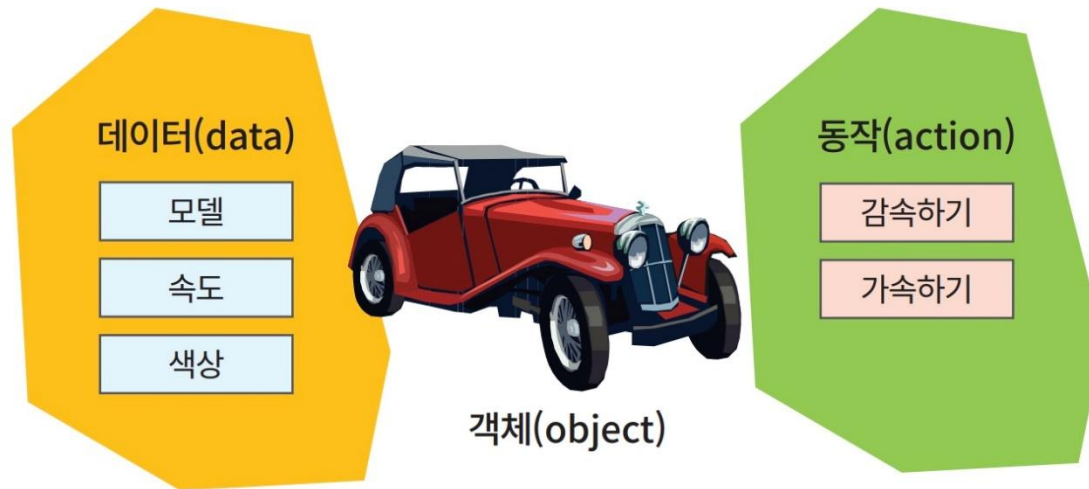
CHAPTER 10.

자바 스크립트 객체



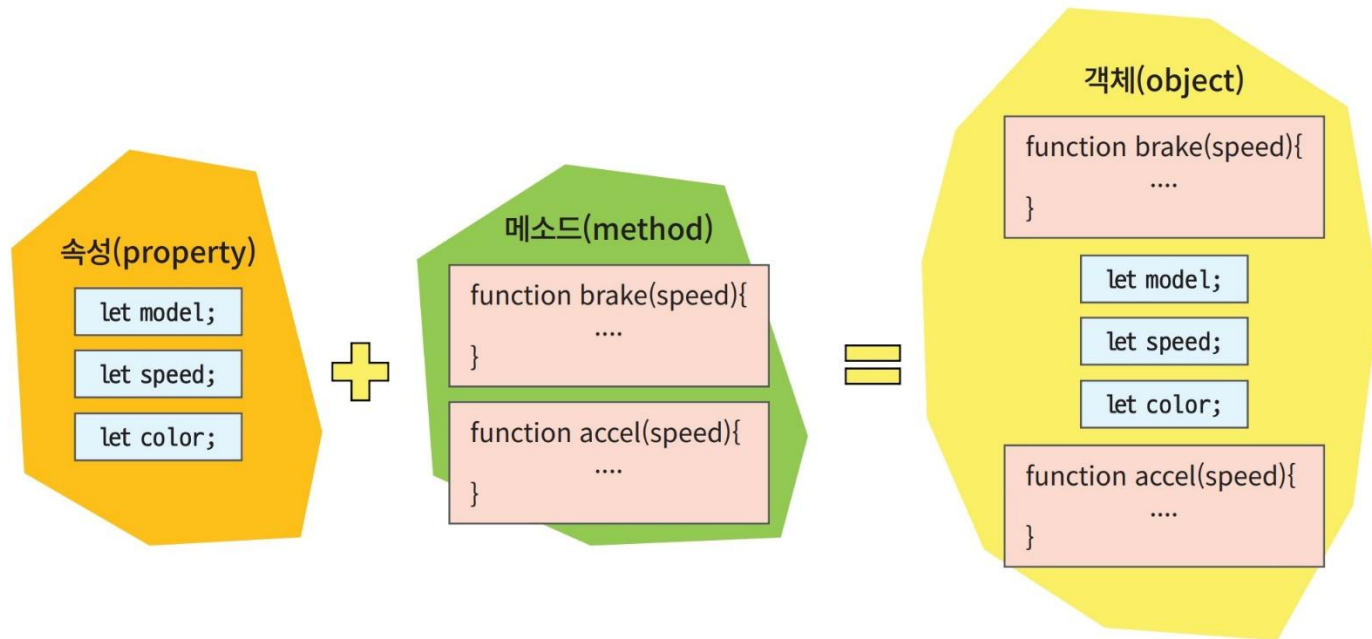
객체

- 객체(object)는 사물의 속성과 동작을 묶어서 표현하는 기법
- (예) 자동차는 메이커, 모델, 색상, 마력과 같은 **데이터**도 있고 출발하기, 정지하기 등의 **동작(기능)**도 가지고 있다.



속성과 메소드

- 객체 안의 데이터를 저장하는 변수를 **속성(property)**라고 하고, 객체 안의 함수를 **메소드(method)**라고 부른다.



객체의 종류

- 객체의 2가지 종류
 - 내장 객체(*bulit-in object*): 생성자가 미리 작성되어 있다.
 - 사용자 정의 객체(*custom object*): 사용자가 생성자를 정의한다.
- 내장 객체들은 생성자를 정의하지 않고도 사용이 가능하다. **Date**, **String**, **Array**와 같은 객체들이 내장 객체이다.

사용자 정의 객체 생성 방법

- 객체를 생성하는 3가지 방법
 - 객체 상수(리터럴 객체)로 직접 생성한다.
 - 생성자 함수를 이용하여 객체를 정의하고 `new`를 통하여 객체의 인스턴스를 생성한다.
 - **(ES6) class** 키워드로 객체의 구조를 설계하고 `new`를 통하여 생성.

#1. 리터럴 객체로 객체 생성

```
var myCar = {  
  model: "520d",  
  speed: 60,  
  color: "red",  
  brake: function () { this.speed -= 10; },  
  accel: function () { this.speed += 10; }  
};
```

객체의 속성

객체의 메소드

```
myCar.color = "yellow";  
myCar.brake();
```



- 객체 멤버들을 사용하기 위해서는 점(dot) 연산자를 사용한다.
먼저 어떤 객체에 속해 있는 지를 말하고 나중에 멤버 이름을 말하는 것이다.

#2. 생성자를 이용한 객체 생성

생성자도 함수
이다.

생성자 이름은 항상 대문자로 한다.

```
function Car(model, speed, color) {
```

this 키워드로
일반 변수와
객체 속성을
구별한다.

```
  this.model = model;
```

```
  this.speed = speed;
```

```
  this.color = color;
```

객체의 속성

```
  this.brake = function () {
```

```
    this.speed -= 10;
```

```
  }
```

```
  this.accel = function () {
```

```
    this.speed += 10;
```

```
  }
```

```
}
```

객체의 메소드

생성자를 이용한 객체 생성

이 변수에
객체가 저장
된다.

new 연산자는 새로운 객체를
만든다.

이 인수들
로 객체가
초기화된다.

```
var myCar = new Car("520d", 60, "white");  
myCar.color = "yellow";  
myCar.brake();
```

객체 생성 예제

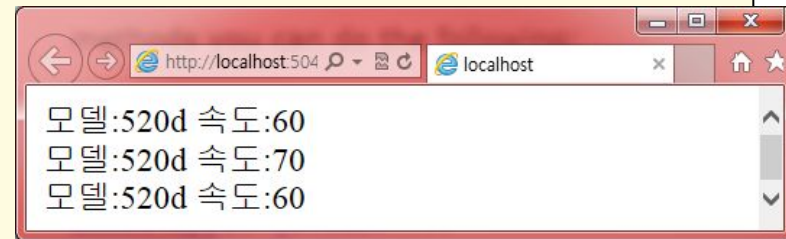
```
<!DOCTYPE html>
<html>
<body>
  <script>
    function Car(model, speed, color)
    {
      this.model=model;
      this.speed=speed;
      this.color = color;

      this.brake = function () {
        this.speed -= 10;
      }
      this.accel = function () {
        this.speed += 10;
      }
    }

    myCar = new Car("520d", 60, "red");
    document.write("모 델:" + myCar.model + " 속도:" + myCar.speed + "<br />");

    myCar.accel();
    document.write("모 델:" + myCar.model + " 속도:" + myCar.speed + "<br />");

    myCar.brake();
    document.write("모 델:" + myCar.model + " 속도:" + myCar.speed + "<br />");
  </script>
</body>
</html>
```



기존 객체에 속성과 메소드 추가

- 기존에 존재하고 있던 객체에도 속성을 추가할 수 있다.
- 생성자 함수는 변경할 필요가 없다.

```
myCar.turbo = true;
myCar.showModel = function() {
    alert( "모델은 " + this.model + "입니다." )
}
```

for/In 루프

- for/in 루프는 객체 안의 모든 속성들에 대하여 어떤 처리를 반복할 수 있는 구조이다.

```
for ( 변수 in 객체 )
```

```
{
```

```
    문장;
```

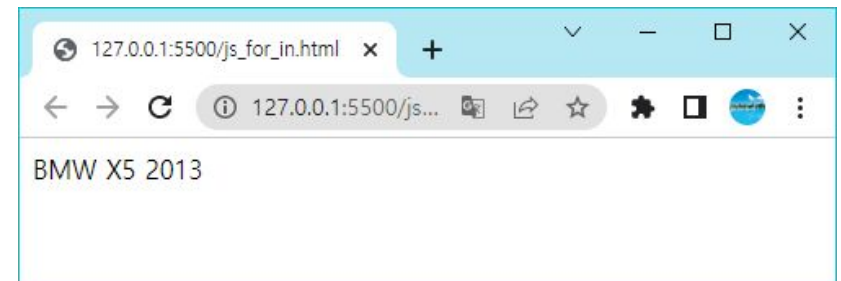
```
}
```

변수에 객체의 속성이
하나씩 대입되면서 반복한다.

예제 소스

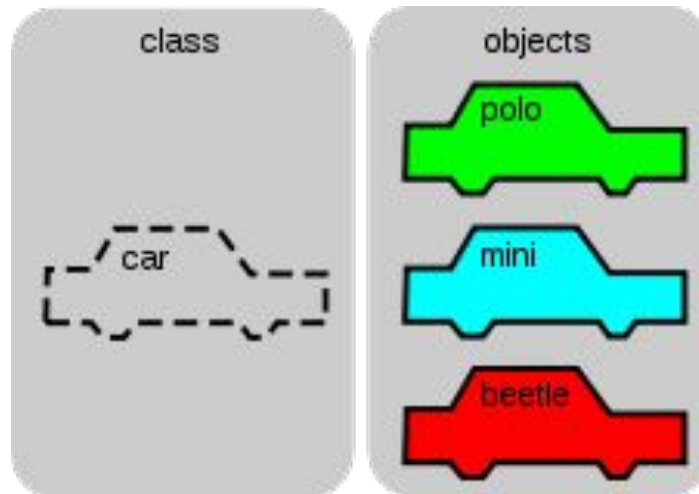
```
<script>
  var myCar = { make: "BMW", model: "X5", year: 2013 };
  var txt="";

  for (var x in myCar) {
    txt += myCar[x] + " ";
  }
  document.write(txt);
</script>
```



#3. class를 이용한 객체 생성

- ECMAScript 2015부터 **class**라고 하는 키워드를 도입하였다. 이것은 파이썬이나 자바 언어의 클래스와 동일한 개념으로 이것을 사용하면 상당히 편리하게 클래스를 정의하고 객체를 찍어낼 수 있다.



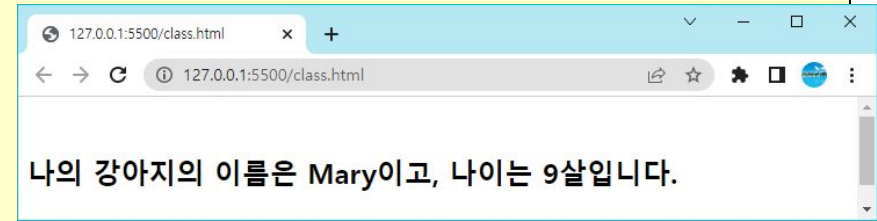
예제 소스

```
<!DOCTYPE html>
<html>
<body>
  <h2 id="test"></h2>

  <script>
    class Dog {
      constructor(name, adoptedYear) {
        this.name = name;
        this.adoptedYear = adoptedYear;
      }

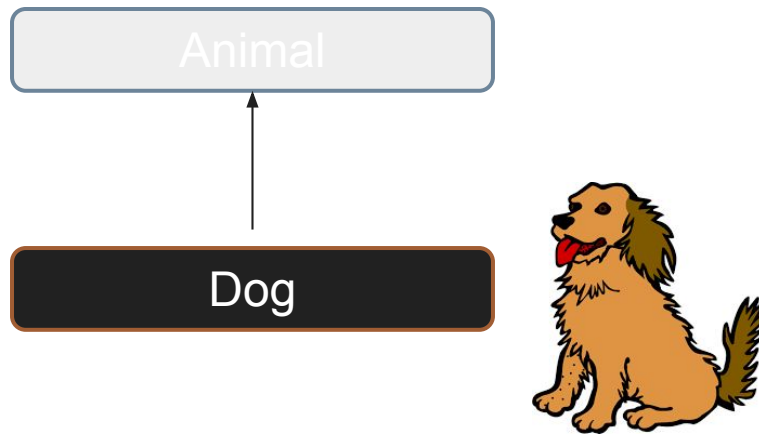
      age() {
        let date = new Date();
        return date.getFullYear() - this.adoptedYear;
      }
    }

    const myDog = new Dog("Mary", 2013);
    document.getElementById("test").innerHTML =
      "나의 강아지의 이름은 " + myDog.name + "이고, 나이는 " + myDog.age() + "살입니다.";
  </script>
</body>
</html>
```



상속이 가능하다.

- 자바스크립트의 새로운 버전에서는 자바 언어와 동일한 방법으로 상속을 사용할 수 있다. 예를 들어서 **Animal**을 상속하여서 **Dog** 클래스를 만들 수 있다.



예제 소스

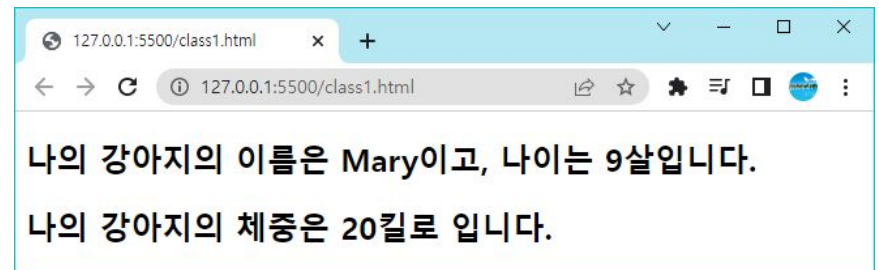
```
<!DOCTYPE html>
<html>
<body>
  <h2 id="test"></h2>
  <h2 id="test1"></h2>
  <script>
    class Animal {
      constructor(weight) {
        this.weight = weight;
      }
    }

    class Dog extends Animal {
      constructor(name, adoptedYear, weight) {
        super(weight);
        this.name = name;
        this.adoptedYear = adoptedYear;
      }
      age() {
        let date = new Date();
        return date.getFullYear() - this.adoptedYear;
      }
    }
  </script>
</body>
</html>
```

예제 소스

```
let myDog = new Dog("Mary", 2013, 20);
document.getElementById("test").innerHTML =
    "나의 강아지의 이름은 " + myDog.name + "이고, 나이는 " + myDog.age() + "살입니다.";

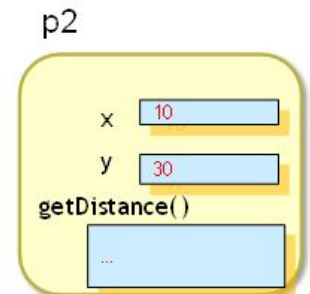
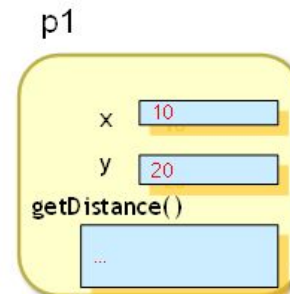
document.getElementById("test1").innerHTML =
    "나의 강아지의 체중은 " + myDog.weight + "킬로 입니다.";
</script>
</body>
</html>
```



프로토타입

- 자바 스크립트에서 메소드를 여러 객체가 공유하려면 어떻게 해야 하는가?
- 현재는 메소드를 공유할 수 없다.

```
function Point(xpos, ypos) {  
    this.x = xpos;  
    this.y = ypos;  
    this.getDistance = function () {  
        return Math.sqrt(this.x * this.x + this.y * this.y);  
    };  
}  
  
var p1 = new Point(10, 20);  
var p2 = new Point(10, 30);
```

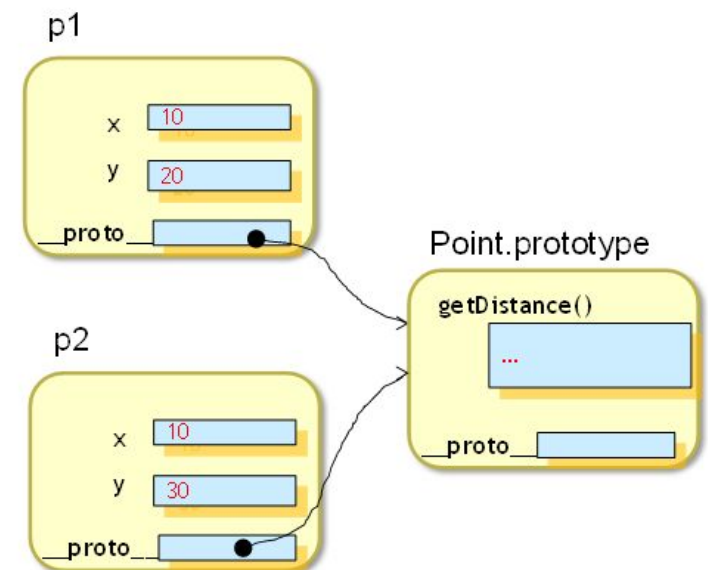


프로토타입

- 자바스크립트의 모든 객체들은 **prototype**이라는 숨겨진 객체를 가지고 있으며 이 객체를 이용하여서 공유되는 메소드를 작성할 수 있다.

```
function Point(xpos, ypos) {  
    this.x = xpos;  
    this.y = ypos;  
}
```

```
Point.prototype.getDistance = function () {  
    return Math.sqrt(this.x * this.x + this.y * this.y);  
};
```



자바 스크립트 내장 객체

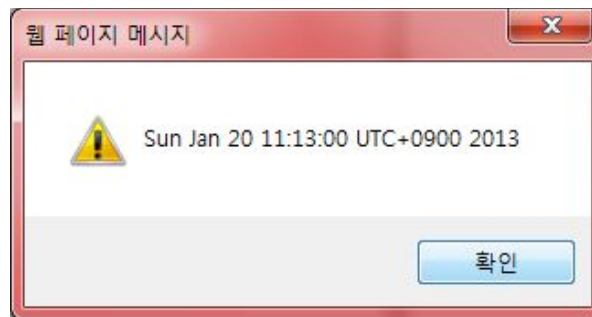
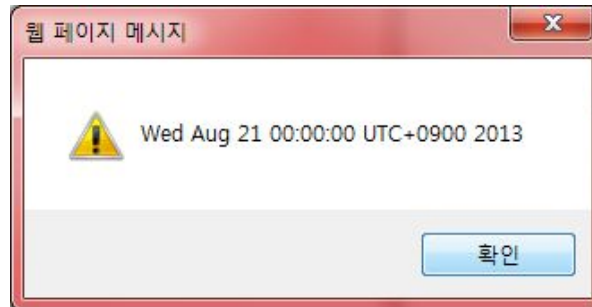
- Date 객체
- Number 객체
- String 객체
- Math 객체
- Array 객체
- ...

Date 객체

- **Date** 객체는 날짜와 시간 작업을 하는데 사용되는 가장 기본적인 객체
 - `new Date()` // 현재 날짜와 시간
 - `new Date(milliseconds)` // 1970/01/01 이후의 밀리초
 - `new Date(dateString)` // 다양한 문자열
 - `new Date(year, month, date[, hours[, minutes[, seconds[, ms]]]])`

예 제

```
<script>  
  var d1 = new Date(2013, 7, 21, 0, 0, 0);  
  var d2 = new Date("January 20, 2013 11:13:00");  
  alert(d1);  
  alert(d2);  
</script>
```



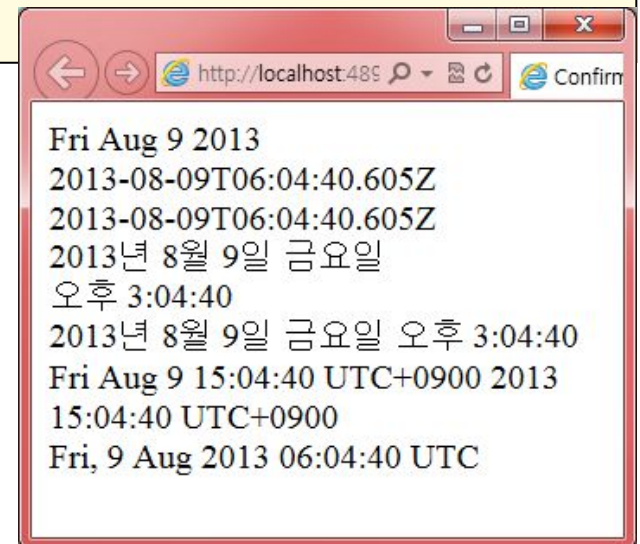
Date 객체의 메소드

- getDate() (1-31 반환)
- getDay() (0-6 반환)
- getFullYear() (4개의 숫자로 된 연도 반환)
- getHours() (0-23 반환)
- getMilliseconds()(0-999)
- getMinutes()(0-59)
- getMonth()(0-11)
- getSeconds()(0-59)

- setDate()
- setDay()
- setFullYear()
- setHours()
- setMilliseconds()
- setMinutes()
- setMonth()
- setSeconds()

예 제

```
<script>
  var today = new Date();
  document.write(today.toString() + "<br>");
  document.write(today.toISOString() + "<br>");
  document.write(today.toJSON() + "<br>");
  document.write(today.toLocaleDateString() + "<br>");
  document.write(today.toLocaleTimeString() + "<br>");
  document.write(today.toLocaleString() + "<br>");
  document.write(today.toString() + "<br>");
  document.write(today.getTimeString() + "<br>");
  document.write(today.toUTCString() + "<br>");
</script>
```



2개의 날짜 비교하기

- **Date** 객체는 2개의 날짜를 비교하는 데도 사용된다. 두 개의 날짜를 비교하는 것은 생각보다 무척 어렵다. 연도, 월, 일, 윤년 등을 따져야 하기 때문이다.



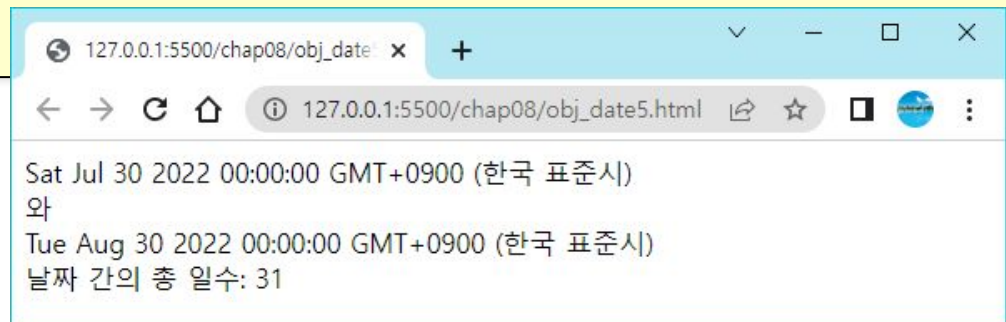
- 자바스크립트에서는 유닉스 운영체제에서 사용하였던 방법을 이용한다. 두 개의 날짜를 비교할 때는 먼저 모든 날짜를 1970년 1월 1일 이후의 밀리초로 변환한다. `getTime()`이라는 메소드를 사용하면 된다.

날짜 비교 예제

- 2022년 7월 30일과 2022년 8월 30일 사이의 총 일수를 계산해보자. 이것을 검사하려면 다음과 같이 한다.

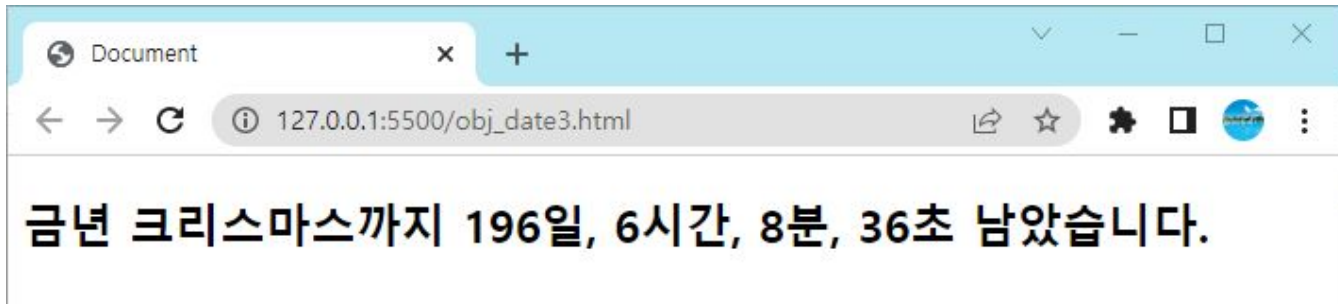
```
<script>
var date1 = new Date("07/30/2022");
var date2 = new Date("08/30/2022");
var diff = date2.getTime() - date1.getTime();
var diff_days = diff / (1000 * 3600 * 24);

document.write(
    date1 + "<br> 와 <br>" + date2 + " <br> 날짜 간의 총 일수: " + diff_days);
</script>
```



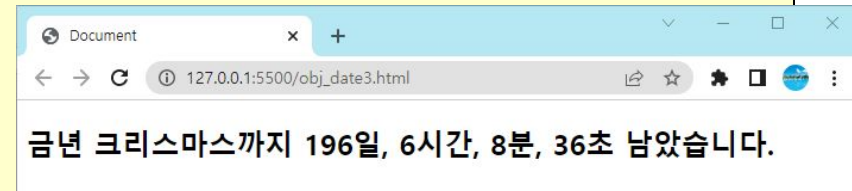
예제: 카운트다운 타이머 만들기

- 지금부터 특정 날짜까지 남은 날짜를 표시하는 코드를 작성하여 보자. 예를 들어서 크리스마스까지, 남은 날짜를 웹 페이지에 표시하는 예제를 작성하여 보자.



예제

```
<!DOCTYPE html>
<body>
  <h2 id='remaining'></h2>
  <script>
    function datesUntilXmas() {
      var now = new Date();
      var newYear = new Date('December 25, ' + (now.getFullYear()));
      var diff = newYear - now;
      var milliseconds = Math.floor(diff % 1000);
      diff = diff / 1000;
      var seconds = Math.floor(diff % 60);
      diff = diff / 60;
      var minutes = Math.floor(diff % 60);
      diff = diff / 60;
      var hours = Math.floor(diff % 24);
      diff = diff / 24;
      var days = Math.floor(diff);
      var outStr = '금년 크리스마스까지 ' + days + '일, ' + hours + '시간, '
        + minutes + '분, ' + seconds + '초' + ' 남았습니다.';
      document.getElementById('remaining').innerHTML = outStr;
      // 1초가 지나면 다시 함수를 호출한다.
      setTimeout("datesUntilXmas()", 1000);
    }
    // 타이머를 시작한다.
    datesUntilXmas();
  </script>
</body>
</html>
```

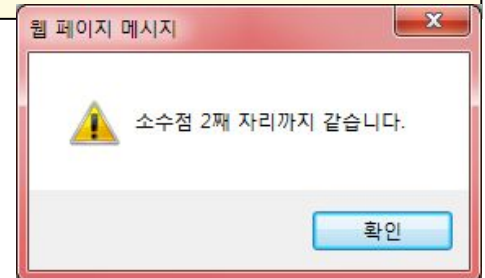


Number 객체

- Number 객체는 수치형 값을 감싸서 객체로 만들어 주는 래퍼(wrapper) 객체
 - `var num = new Number(7);`
- 메소드
 - `toFixed([digits])`
 - `var num = 123.456789;`
 - `document.writeln(num.toFixed(1) + '<br>'); // 123.5`
 - `toPrecision([precision])`
 - `var num = 123.456789;`
 - `document.writeln(num.toPrecision(1) + '<br>'); // 1e+2`
 - `toString([radix])`

예 제

```
<script>
  var count1, count2;
  count1 = new Number(1.237);
  count2 = 1.238;
  if (count1.toFixed(2) === count2.toFixed(2))
    alert("소수점 2째 자리까지 같습니다.");
</script>
```



String 객체

- String 객체는 텍스트를 저장하고 조작하는 객체로 아주 많이 사용된다. String 클래스의 메소드를 잘 알아두어야 편리하다.
- 웹 프로그래밍에서도 텍스트를 조작해야 하는 경우가 상당히 많기 때문이다.
- 자바스크립트에서 문자열(String)에는 2가지 종류가 있다.
 - 문자열 리터럴은 따옴표로 생성된다.
 - 문자열 객체는 키워드 **new**를 이용하여 생성된다.

```
var s1 = "ABCD";  
var s2 = new String("ABCD");
```

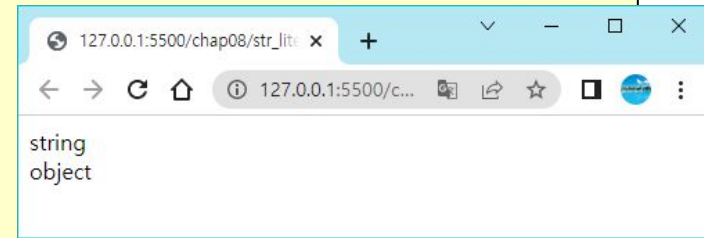
예제

```
<script>

var sLiteral = "문자열 리터럴";

var sObject = new String("문자열 객체");

document.writeln(typeof (sLiteral) + '<br>'); // string
document.writeln(typeof (sObject) + '<br>'); // object
</script>
```



- 그런데 문자열 리터럴도 **String** 객체의 모든 속성과 메소드를 사용할 수 있는데, 자바스크립트가 임시적으로 문자열 리터럴을 문자열 객체로 형변환을 하기 때문이다.

```
var sLiteral = "문자열 리터럴";
sLiteral = sLiteral.concat('123');
document.writeln(sLiteral + '<br>'); // 문자열 리터럴123
```

문자열의 길이와 개별 문자 접근

- String 객체의 속성 `length`는 문자열의 길이를 나타낸다.

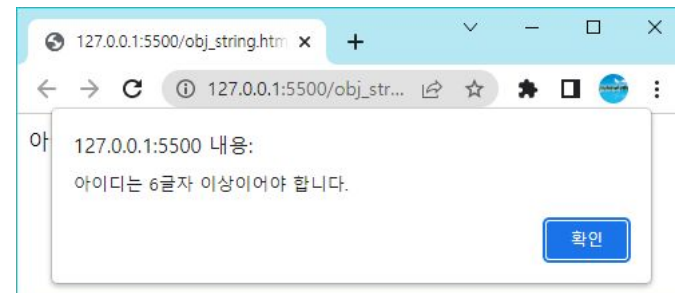
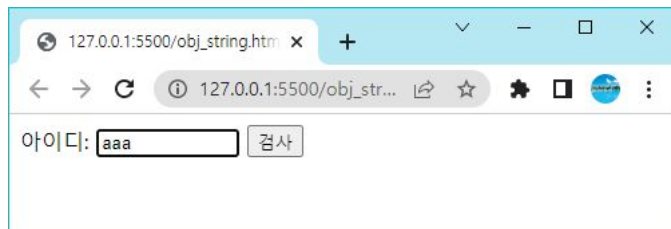
```
var s = "abcdef";  
var length = s.length;    // 6이 된다.
```

- 문자열 안의 개별적인 문자에 접근하려면 배열처럼 `[]` 안에 인덱스를 넣으면 된다. 인덱스는 **0**부터 시작한다.

```
var s = "abcdef";  
var ch = s[1];            // ch는 'b'가 된다.
```

예제

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function checkID() {
      var s = document.getElementById("id").value;
      if (s.length < 6)
        alert("아이디는 6글자 이상이어야 합니다.");
    }
  </script>
</head>
<body>
  아이디: <input type="text" id="id" size=10>
  <button onclick="checkID()">검사</button>
</body>
</html>
```



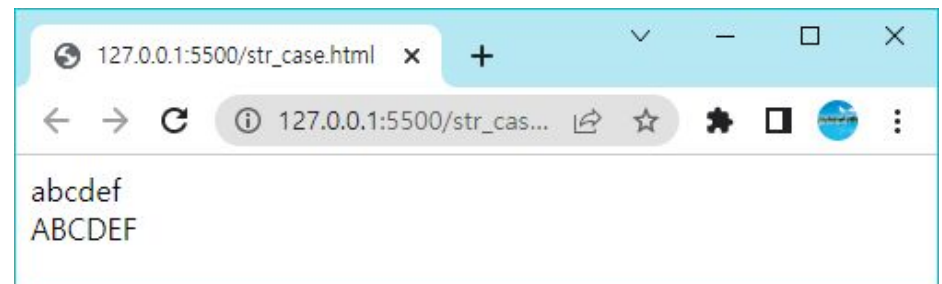
String 객체 메소드

방법	설명
charAt(위치)	지정된 위치의 문자를 반환한다.
indexOf(검색 문자열, 위치)	지정된 숫자 인덱스에서 시작하여 지정된 문자열이 처음 나타나는 인덱스를 반환한다. 찾을 수 없으면 -1을 반환한다.
replace(검색값, 대체값)	지정된 값을 검색하고 대체값으로 바꾸고 새 문자열을 반환한다.
match(RegExp)	지정된 정규식을 기반으로 일치하는 항목을 검색한다.
slice(시작 위치, 종료 위치)	지정된 시작 및 끝 인덱스를 기반으로 문자열의 일부를 추출하고 새 문자열을 반환한다.
split(분리자, 제한 길이)	지정된 분리자에 따라 문자열을 문자열 배열로 분할한다.
substr(시작, 길이)	지정된 시작 위치에서 지정된 문자 수(길이)까지 문자열의 일부를 반환한다.
substring(시작, 끝)	시작 인덱스와 끝 인덱스 사이의 문자열에서 부분 문자열을 반환한다.
toLowerCase()	소문자로 변환한다.
toUpperCase()	대문자로 변환한다.
valueOf()	문자열을 수치값으로 변환한다.

대소문자 변환

- toUpperCase()와 toLowerCase()를 사용하면 문자열의 글자들을 대문자, 혹은 소문자로 변환할 수 있다.

```
<script>
  var s = 'aBcDeF';
  var result1 = s.toLowerCase();
  var result2 = s.toUpperCase();
  document.write(result1 + "<br>");
  document.write(result2 + "<br>");
</script>
```



문자열 비교

- 문자열을 비교하는 가장 쉬운 방법은 `==` 연산자를 사용하는 것이다.

```
if( "apple" == "apple" ){           // true
```

```
...
```

```
}
```

- 문자열의 사전적인 순서를 알려면 `<이나 >` 연산자를 사용한다.

```
if( "cat" < "dog" ){               // true
```

```
...
```

```
}
```

문자열 검색

- `indexOf()` 메소드는 문자열 안에서 주어진 텍스트가 처음 등장하는 위치를 반환한다. 아주 편리한 메소드이다.

```
<script>
  var s = "A bad workman always blames his tools.";
  var n = s.indexOf("workman");
  document.write(n + '<br>'); // 6
</script>
```



문자열 매칭

- match() 메소드에서는 정규식(regular expression)을 사용할 수 있다.

```
<script>
```

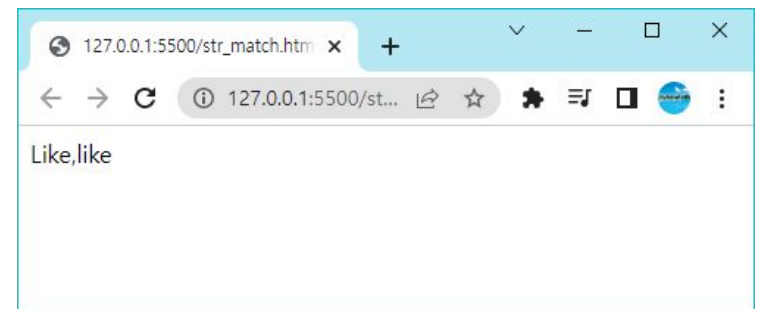
```
let str = 'Like father, like son.';
```

```
// like를 찾는다. i와 g는 옵션으로 insensitive, globally를 의미한다.
```

```
result = str.match(/like/ig);
```

```
document.write(result + '<br>');
```

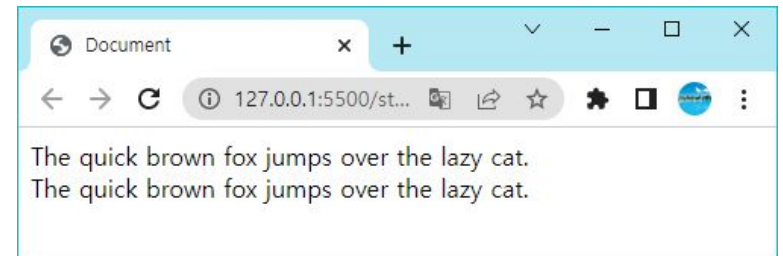
```
</script>
```



문자열 대체

- `replace()` 메소드는 문자열 안에서 주어진 값을 다른 값으로 대체한다. `replace()`도 정규식을 사용할 수 있어서 강력한 기능 중의 하나이다.

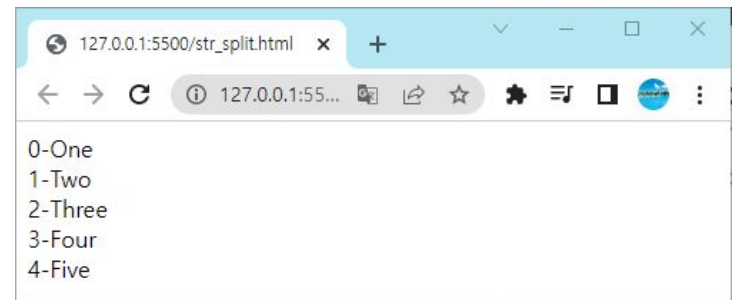
```
<script>
  var p = 'The quick brown fox jumps over the lazy dog.';
  document.write(p.replace('dog', 'cat')+"<br>");
  var regex = /Dog/i;
  document.write(p.replace(regex, 'cat')+"<br>");
</script>
```



문자열 분할

- `split()`는 첫 번째 인수를 분리자로 하여서 주어진 문자열을 분리한 후에 각 항목들을 가지고 있는 배열을 반환한다

```
<script>
  var s = "One,Two,Three,Four,Five";
  array = s.split(',');
  for (i = 0; i < array.length; i++) {
    document.write(i + '-' + array[i] + '<br>');
  }
</script>
```



공백 제거

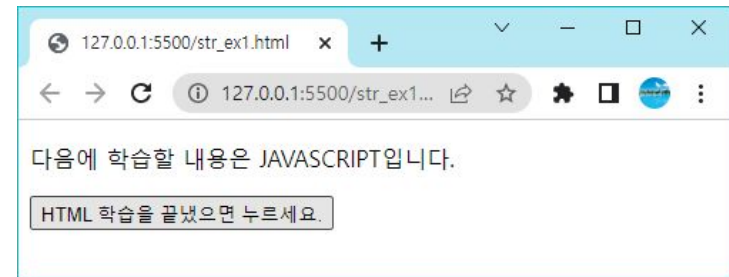
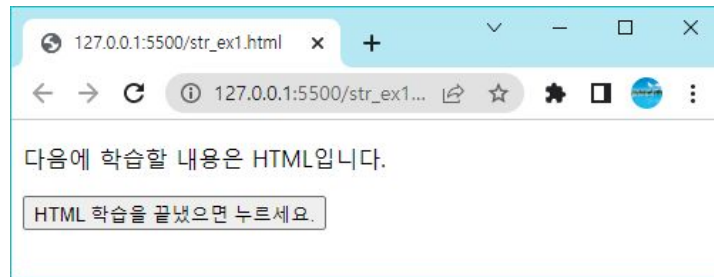
- 문자열 앞뒤에 붙은 공백 문자들을 제거하는데는 `trim()` 메소드를 사용할 수 있다.

```
var s = " Hello world! ";  
s = s.trim();           // s="Hello World!"
```

- `trim()` 메소드가 왜 필요할까? 사용자가 문자열을 입력할 때 실수로 앞뒤에 공백을 추가하는 일도 매우 흔하게 발생하기 때문이다.

예제: replace 메소드 이용

- 버튼을 누르면 문단의 "HTML"이 "JAVASCRIPT"로 변경되는 웹페이지를 만들어보자.



```
<!DOCTYPE html>
<html>
<body>
  <p id="test">다음에 학습할 내용은 HTML입니다. </p>
  <button onclick="myFun()">HTML 학습을 끝냈으면 누르세요. </button>
  <script>
    function myFun() {
      let text = document.getElementById("test").innerHTML;
      document.getElementById("test").innerHTML = text.replace("HTML", "JAVASCRIPT");
    }
  </script>
</body>
</html>
```

Math 객체

속성	설명
<u>E</u>	오일러의 상수 (약 2.718)
<u>LN2</u>	자연 로그(밑수: 2) (약 0.693)
<u>LN10</u>	자연 로그(밑수:10) (approx. 2.302)
<u>PI</u>	파이 상수 (약 3.14)
<u>SQRT1_2</u>	1/2의 제곱근(약 0.707)
<u>SQRT2</u>	2의 제곱근 (약 1.414)

메소드	설명
<u>abs(x)</u>	절대값
<u>acos(x)</u> , <u>asin(x)</u> , <u>atan(x)</u>	아크 삼각함수
<u>ceil(x)</u> , <u>floor(x)</u>	실수를 정수로 올림, 내림 함수
<u>cos(x)</u> , <u>sin(x)</u> , <u>tan(x)</u>	삼각함수
<u>exp(x)</u>	지수함수
<u>log(x)</u>	로그함수
<u>max(x,y,z,...,n)</u>	최대값
<u>min(x,y,z,...,n)</u>	최소값
<u>pow(x,y)</u>	지수함수 x^y
<u>random()</u>	0과 1 사이의 난수값 반환
<u>round(x)</u>	반올림
<u>sqrt(x)</u>	제곱근

계산기 예제

```
<html>

<head>
  <script>
    function calc(type) {
      x = Number(document.calculator.number1.value);
      if (type == 1)
        y = Math.sin((x * Math.PI) / 180.0);
      else if (type == 2)
        y = Math.log(x);

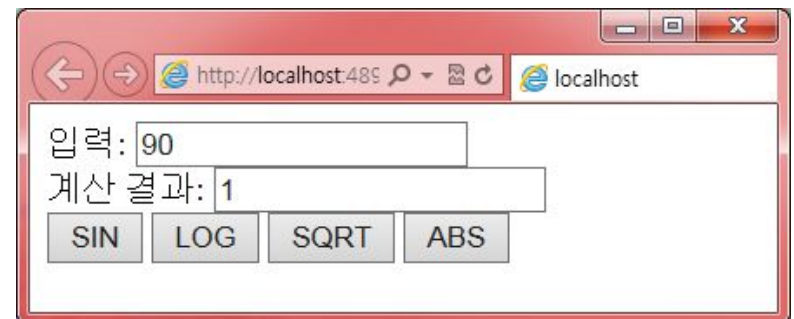
      else if (type == 3)
        y = Math.sqrt(x);
      else if (type == 4)
        y = Math.abs(x);
      document.calculator.total.value = y;
    }
  </script>
</head>
```

예 제

```
<body>
  <form name="calculator">
    입력:      <input type="text" name="number1"><br />
    계산 결과:  <input type="text" name="total"><br />
    <input type="button" value="SIN" onclick="calc(1);">
    <input type="button" value="LOG" onclick="calc(2);">
    <input type="button" value="SQRT" onclick="calc(3);">

    <input type="button" value="ABS" onclick="calc(4);">
  </form>

</body>
</html>
```



난수 발생

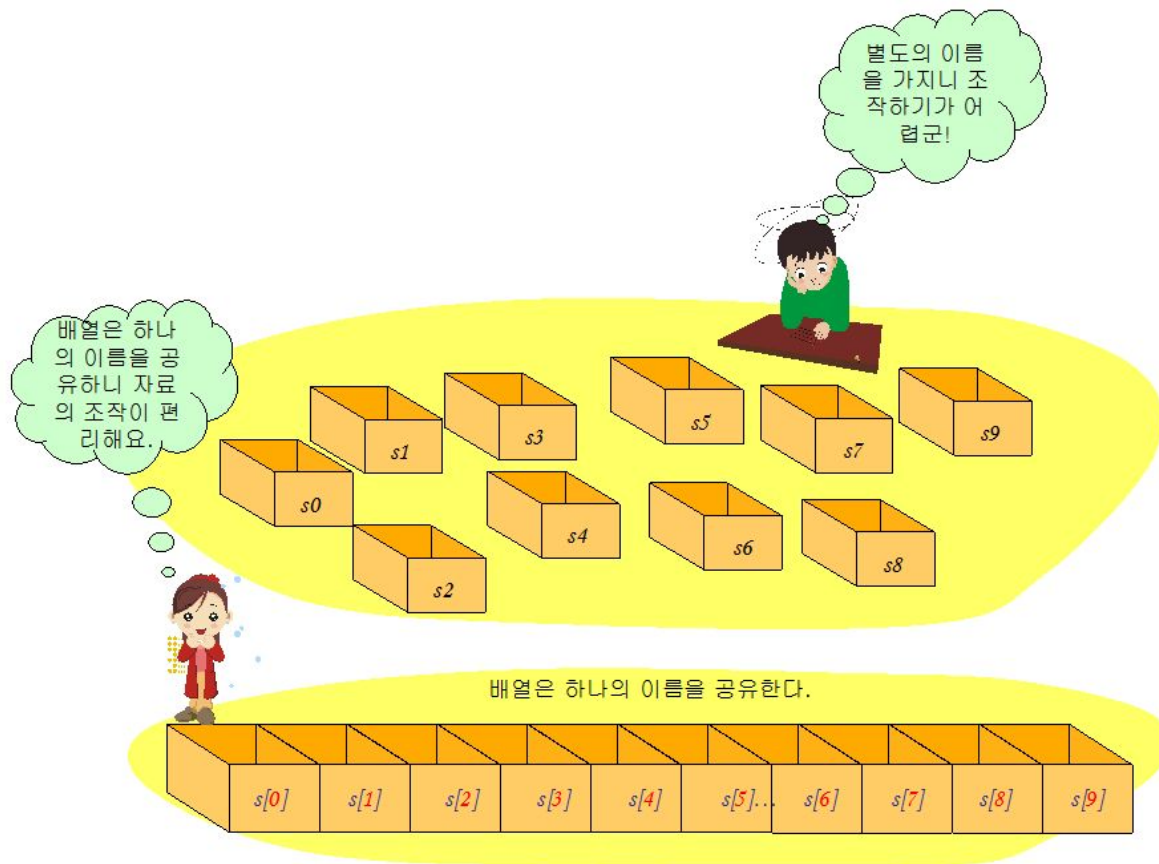
- `Math.random()`을 호출하면 0에서 1 미만의 난수가 발생된다. 따라서 여기에 적당한 수를 곱해서 사용하면 된다.

```
var f = Math.random() * 100; // f는 0.0~100.0 미만의 실수  
var n = Math.floor(f);       // n은 0~99사이의 정수
```



Array 배열 객체

- 많은 값을 저장할 수 있는 공간이 필요할 때 배열을 사용한다.
- 서로 관련된 데이터를 차례로 접근하여서 처리할 수 있다.



배열을 생성하는 2가지 방법

- 리터럴로 배열 생성
 - `var fruits = ["apple", "banana", "peach"];`
- Array 객체로 배열 생성
 - `var fruits=new Array("apple","banana","orange");`

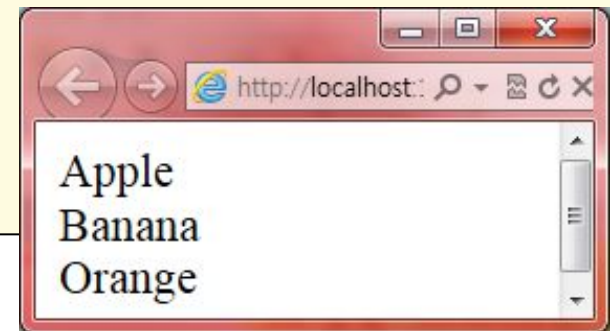
예제

```
<!DOCTYPE html>
<html>
<body>

<script>
    var i;
    var fruits = new Array();
    fruits[0] = "Apple";
    fruits[1] = "Banana";
    fruits[2] = "Orange";

    for (i = 0; i < fruits.length; i++) {
        document.write(fruits[i] + "<br>");
    }
</script>

</body>
</html>
```



자바스크립트 배열의 특징

- 배열의 크기가 자동으로 조절된다. 현재 배열의 크기보다 큰 인덱스를 사용하면 자동으로 증가한다.

```
let myArray = new Array();  
myArray[0] = "apple";  
myArray[99] = "banana";
```

- 자바스크립트에서는 하나의 배열에 여러 가지 자료형을 혼합하여서 저장할 수 있다.

```
let myArray = new Array();  
myArray[0] = "apple";           // 문자열 저장  
myArray[1] = new Date();       // 객체 저장  
myArray[2] = 3.14;             // 실수 저장
```

연관 배열

```
var arr= new Array();
```

```
arr['aa']=10;
```

```
arr['bb']=20;
```

```
arr['cc']=30;
```

```
document.write( arr['aa']+"<br/>" );
```

```
document.write( arr['bb']+"<br/>" );
```

```
document.write( arr['cc']+"<br/>" );
```

```
document.write('<hr/>');
```

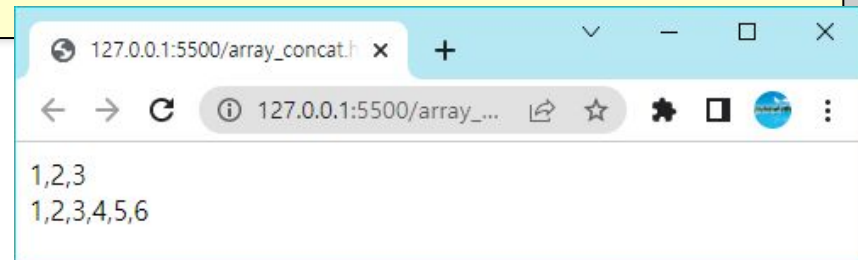
Array 객체의 메소드

- 속성
 - length, prototype
- 메소드
 - concat()
 - indexOf()
 - join()
 - lastIndexOf()
 - pop()
 - push()
 - shift()
 - slice()
 - sort()
 - splice()

Array 메소드: concat()

- `concat()` 메소드는 전달된 인수를 배열의 끝에 추가한다. 인수는 배열일 수도 있다.

```
<script>
  var x = [1, 2, 3];
  var y = [4, 5, 6];
  var joined = x.concat(y);
  document.writeln(x+"<br>"); // 출력: 1,2,3
  document.writeln(joined+"<br>"); // 출력: 1,2,3,4,5,6
</script>
```



Array 메소드: indexOf()

- indexOf() 메소드는 요소의 값을 가지고 요소의 인덱스를 찾을 때, 사용한다.

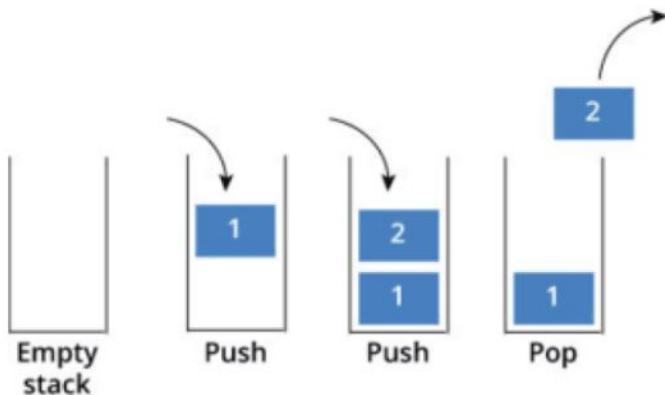
```
<script>  
  var fruits = ["apple", "banana", "grape"];  
  document.writeln(fruits.indexOf("banana"));    // 출력: 1  
</script>
```

Array 메소드: push(), pop()

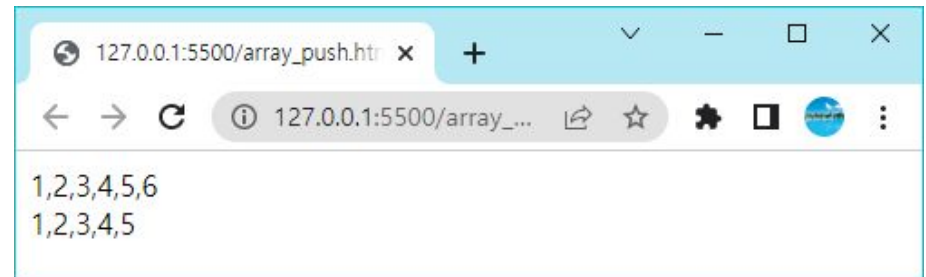
- `push()`는 스택(stack)에 데이터를 삽입하는 메소드이다. `pop()`은 스택에서 데이터를 꺼내는 메소드이다.

```
<script>
  var numbers = [1, 2, 3, 4, 5];

  numbers.push(6);
  document.writeln(numbers + '<br>');    // 출력: 1,2,3,4,5,6
  item = numbers.pop();
  document.writeln(numbers + '<br>');    // 출력: 1,2,3,4,5,
</script>
```



Stack

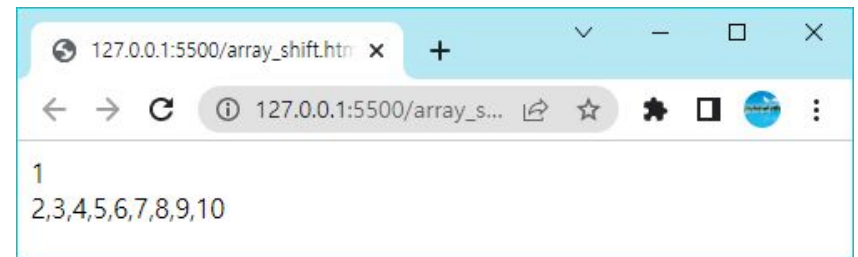


Array 메소드: shift()

- shift()는 큐(queue)에서 데이터를 꺼내는 메소드이다.

```
<script>
  var numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

  var item = numbers.shift();
  document.writeln(item + '<br>');    // 출력: 1
  document.writeln(numbers + '<br>'); // 출력: 2,3,4,5,6,7,8,9,10
</script>
```



Queue

자료구조 Stack vs Queue

Stack
스택



LIFO
LAST IN FIRST OUT

Queue
큐



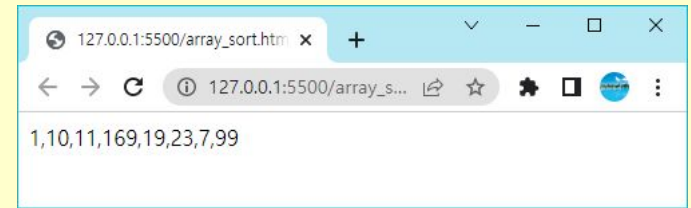
FIFO
FIRST IN FIRST OUT

VS

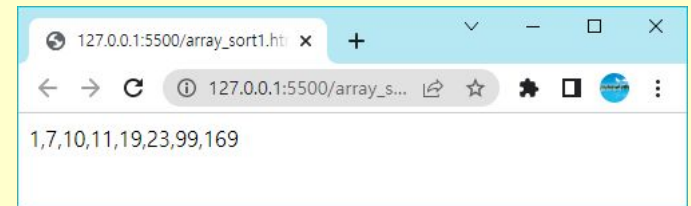
Array 메소드: sort()

- `sort()` 메소드는 배열을 알파벳 순으로 정렬한다. 숫자를 정렬하면 우리의 예상과 다를 수 있다.

```
<script>
  var myArray = [10, 7, 23, 99, 169, 19, 11, 1];
  myArray.sort()
  document.writeln(myArray);
</script>
```



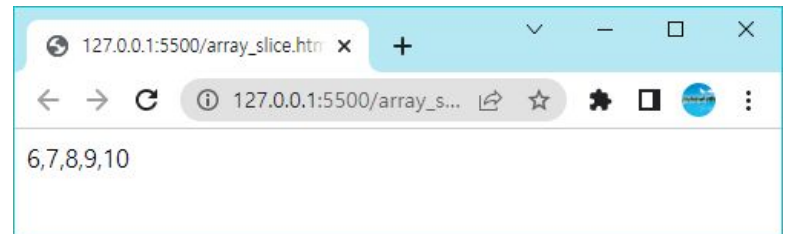
```
<script>
  var myArray = [10, 7, 23, 99, 169, 19, 11, 1];
  myArray.sort(function (a, b) { return a - b });
  document.writeln(myArray);
</script>
```



Array 메소드: slice()

- slice() 메소드는 배열의 일부를 복사하여 새로운 배열로 반환한다.

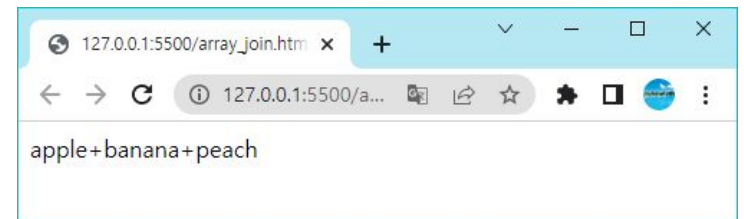
```
<script>  
  var numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  var newArray = numbers.slice(5);  
  document.writeln(newArray + '<br>');    // 출력: 6,7,8,9,10  
</script>
```



Array 메소드: join()

- join()은 배열의 요소들을 하나의 문자열로 출력한다.

```
<script>
  var fruits = ["apple", "banana", "peach" ];
  var s = fruits.join("+");
  document.writeln(s+'<br>'); // 출력: apple+banana+peach
</script>
```

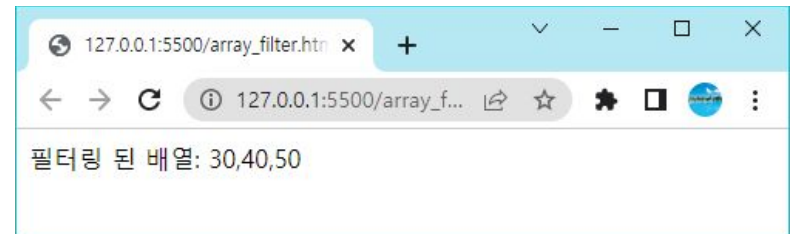


Array 메소드: filter()

- `filter()` 메소드는 어떤 조건에 부합하는 요소만을 선택하여 새로운 배열로 만들어서 반환한다.

```
<script>
  var numbers = [10, 20, 30, 40, 50];
  function isBigEnough(element, index, array) {
    return (element >= 30);
  }

  var filtered = numbers.filter(isBigEnough);
  document.write("필터링 된 배열: " + filtered);
</script>
```



2차원 배열

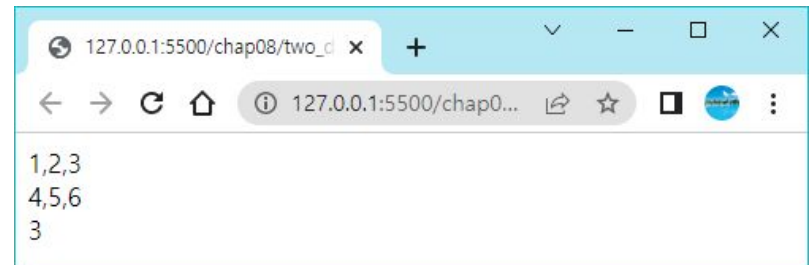
- 자바스크립트에서는 2차원 배열이 가능한가? 가능하다!

```
var z = [ [1, 2, 3], [4, 5, 6] ]
```

```
document.writeln(z[0] + "<br>"); // 출력: 1, 2, 3
```

```
document.writeln(z[1] + "<br>"); // 출력: 4, 5, 6
```

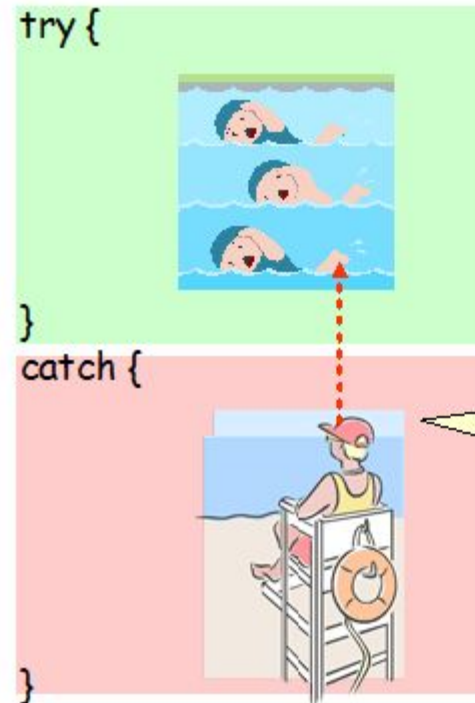
```
document.writeln(z[0][2] + "<br>"); // 출력: 3
```



오류 처리

- 자바스크립트에서의 예외 처리기는 **try** 블록과 **catch** 블록으로 이루어진다.

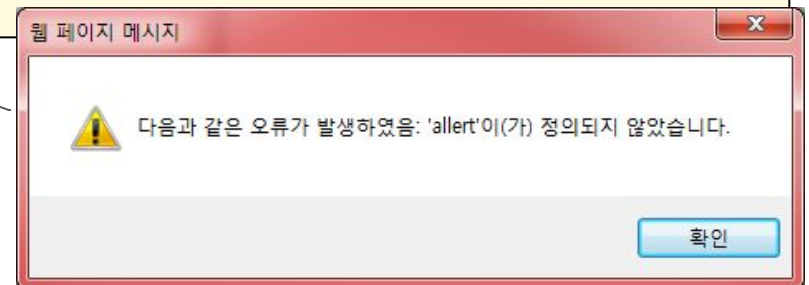
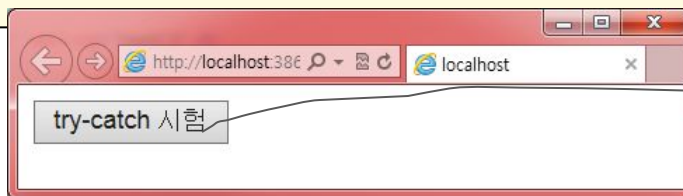
```
try  
{  
    // 예외가 발생할 수 있는 코드  
}  
  
catch (변수)  
{  
    // 예외를 처리하는 코드  
}
```



try블록에서 오류가 발생하면 처리합니다.

예제

```
<!DOCTYPE html>
<html>
<head>
  <script>
    var msg = "";
    function test() {
      try {
        alert("Hello World!");
      }
      catch (error) {
        msg = "다음과 같은 오류가 발생하였음: " + error.message;
        alert(msg);
      }
    }
  </script>
</head>
<body>
  <input type="button" value="try-catch 시험" onclick="test()" />
</body>
</html>
```



throw 문장

- **throw** 문장은 개발자가 오류를 생성할 수 있도록 한다.
- **throw** 문장을 사용하여 오류 처리를 이용할 수도 있다.
 - (예) 숫자 맞추기 게임

예제

```
<!DOCTYPE html>
<html>
<body>
  <script>
    var solution = 53;
    function test() {
      try {
        var x = document.getElementById("number").value;
        if (x == "") throw "입력없음";
        if (isNaN(x)) throw "숫자가 아님";
        if (x > solution) throw "너무 큼";
        if (x < solution) throw "너무 작음";
        if (x == solution) throw "성공";
      }
      catch (error) {
        var y = document.getElementById("message");
        y.innerHTML = "힌트: " + error;
      }
    }
  </script>
```

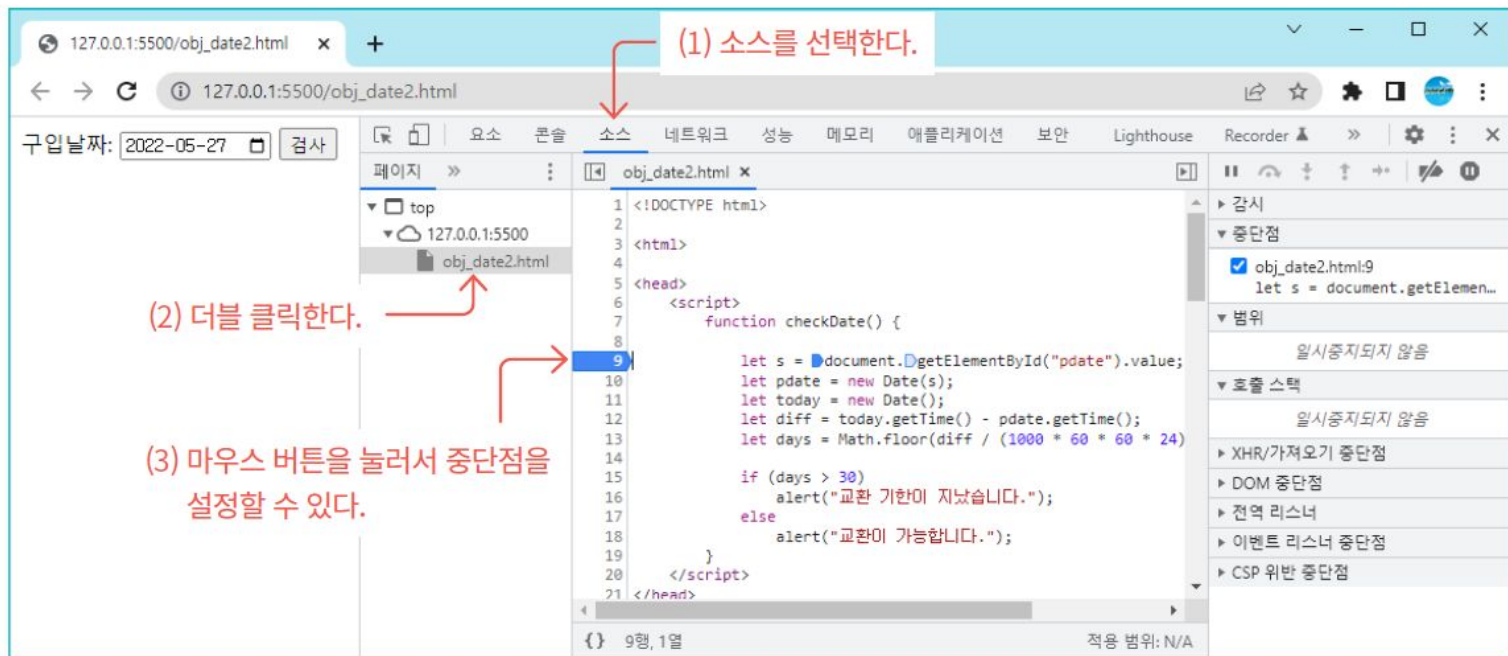
예 제

```
<h1>Number Guess</h1>
  <p>1부터 100 사이의 숫자를 입력하십시오.</p>
  <input id="number" type="text">
  <button type="button" onclick="test()">숫자 추측</button>
  <p id="message"></p>
</body>
</html>
```



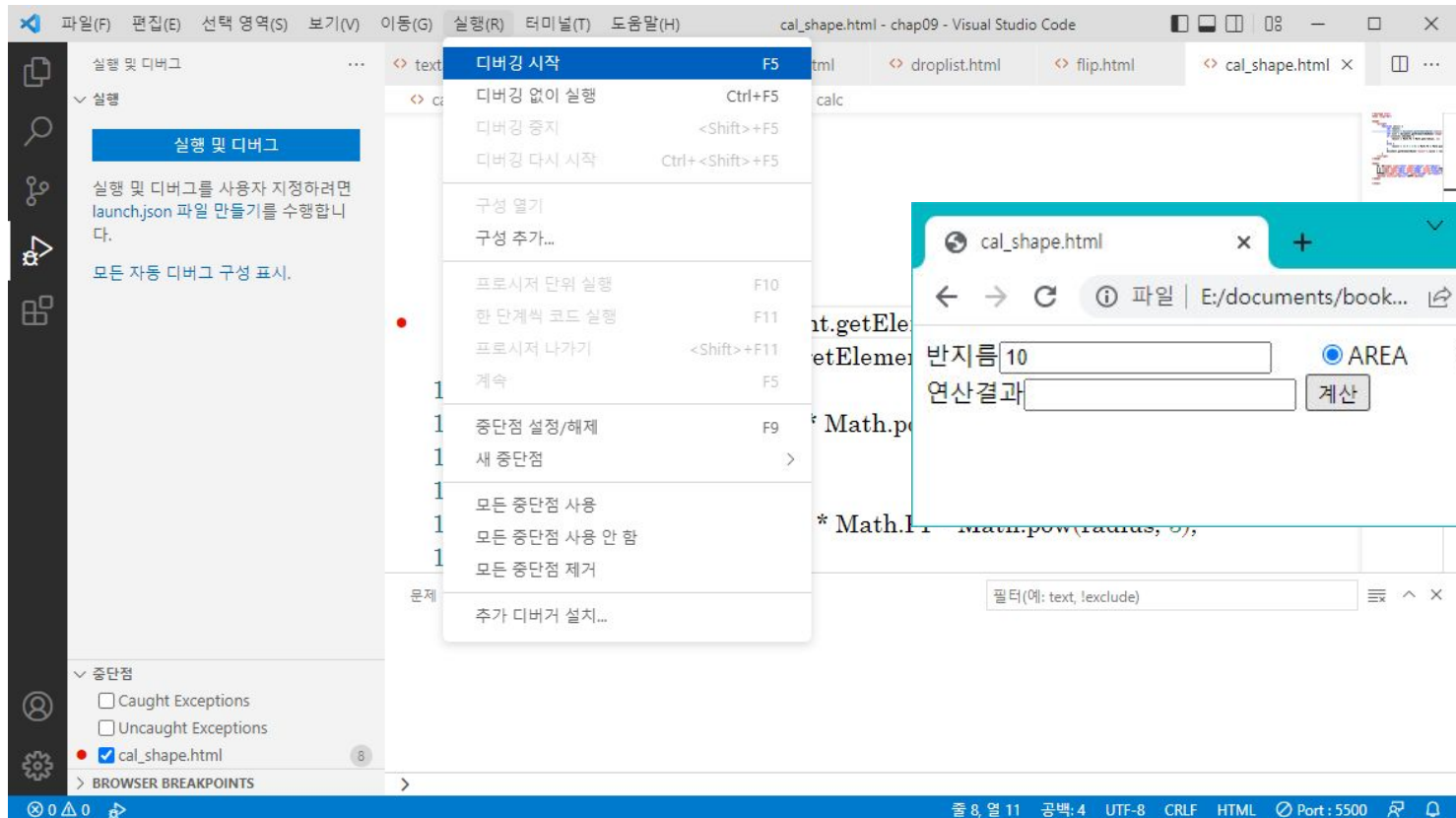
자바스크립트 디버깅하기(크롬)

- 이때에는 크롬의 개발자 도구를 이용하여 디버깅하면 편리하다. 다음과 같은 디버깅 도구들을 사용하도록 하자. 웹페이지에서 마우스 오른쪽 버튼을 누르고 “검사”를 선택하면 개발자 모드가 나타난다.



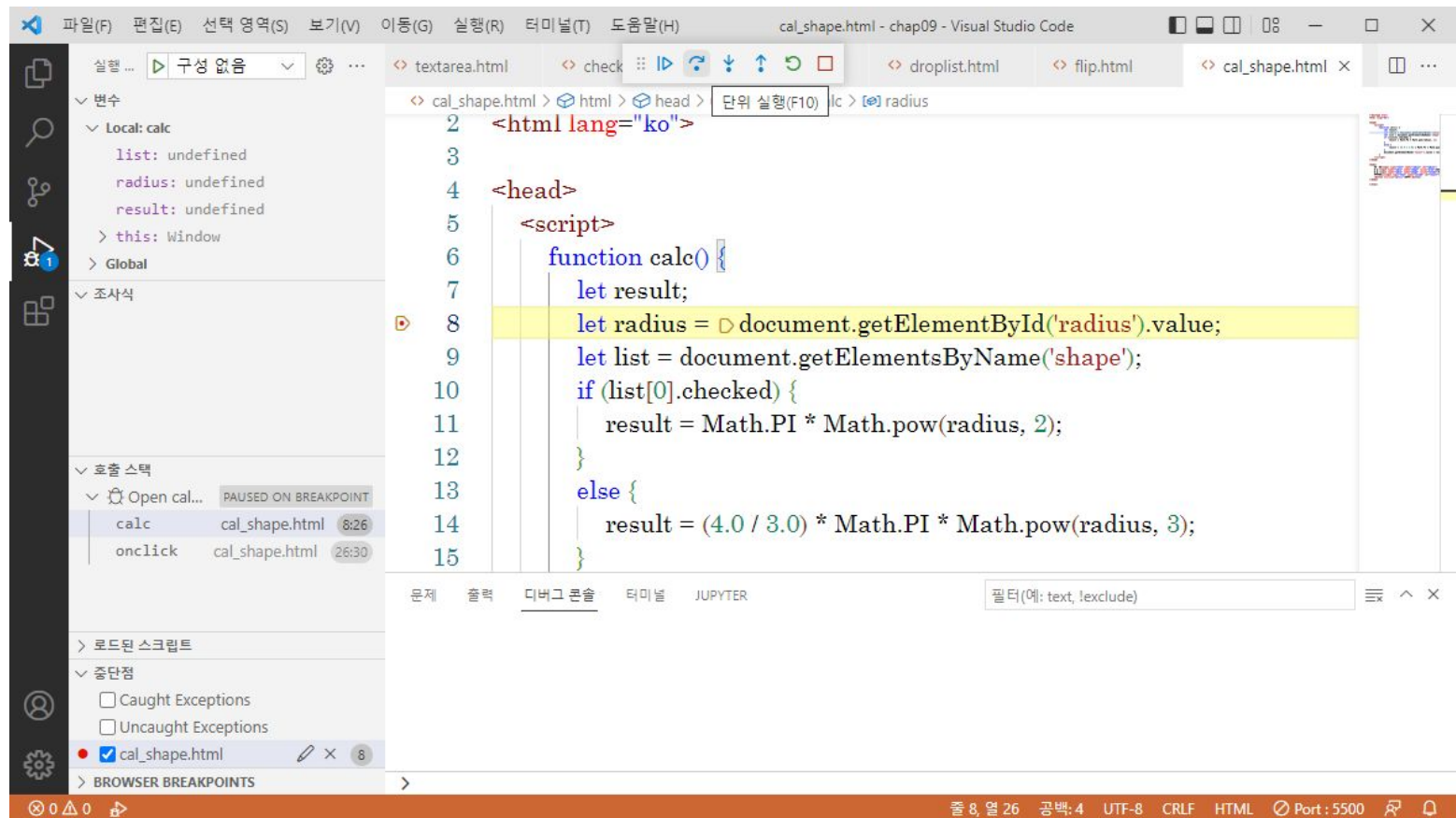
자바스크립트 디버깅 방법(VScode)

- [실행]->[디버깅 시작]을 누르고 웹페이지가 나타나면 값을 입력한다.



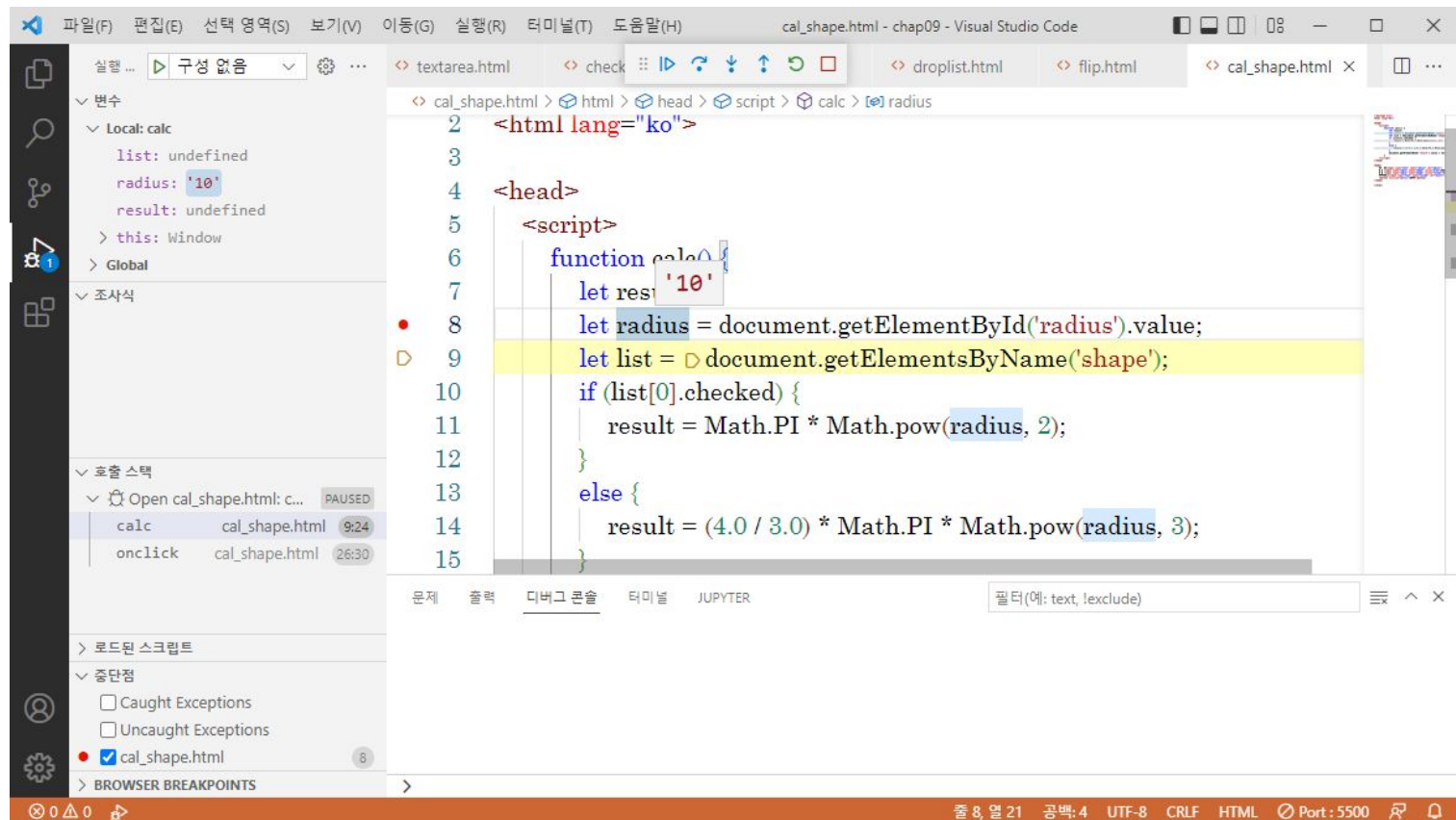
자바스크립트 디버깅 방법(VScode)

- F10을 눌러서 다음 문장으로 간다.



자바스크립트 디버깅 방법(VScode)

- 변수 위에 마우스 커서를 놓으면 변수의 값을 알 수 있다.



Q & A

